


# MSc in Artificial Intelligence and Machine Learning

# CS4096 Artificial Intelligence for Games

## Final Project Report (Treasure Hunt Pirate Game)

**Module Leader:** Prof. Gavin Wade
**Student Name**: Simran Kukreja
**Student ID**: 25141104
**Word Count**: 1290
**Link to project demonstration video**: https://youtu.be/BKwtpCQlaqQ
**Link to project files:** Project files
**Link to source code:** Source Code

# Introduction

We (me along with my teammate Jai Parakh ) have created a Treasure Hunt Pirate Game. There are 3 Pirate characters, one of them is the Player and the other two are Enemy AI's, all three are followed by a Wolf companion. The game is set up on an island, where there exist treasure chests scattered all over and the companion can sniff the treasure and tell his pirate(user or enemy) about it. When a pirate reaches the treasure and digs it up, he will be transferred all the coins that are present in the chest. Also, when two pirates whether user and enemy AI or both the enemies AI encounter each other, they will enter into a combat mode i.e., they will attack each other and one of them dies based on his combat power with the dead pirate's coins being transferred to the opponent who killed him.
The game starts with both of them roaming with their companion and digging up treasure based on their companion's signals. Both the enemy AI's are randomly assigned health, aggression and greediness.

I worked on Enemy AI's functionality, where I implemented two major AI techniques namely **Finite State Machine(FSM)** and **Decision Making.** Other major components I worked upon are the **Game Map,** and **Pathfinding** which is implemented using **Unity's NavMesh**.

I have built the game map including the terrain, its path textures and trees using the free assets from Unity Asset Store, ADG_Textures [1], LowPolyTropicalEnvironment [2], NatureStarterKit2 [3]. Other assets used for the game are StylizedCharacter(for pirate character) [4], Free medieval weapons(for pirate weapon) [5], Animals_FREE(for Wolf companion) [6].

# Analysis

This section explains AI techniques used in the game:

**Decision Making** which is based on the random personality traits assigned to the user. In the game, decision making is implemented using three parameters initialised randomly.

Health - within a range of 30 to 100
Aggression - within a range of 0.3 to 0.9
Greediness - within a range of 0.3 to 0.9.

Use case of these parameters is as follows:

- Health and aggression:
  The health and aggression together determine the combat power = health * aggression of the pirate. After the pirates attack each other, the one with lower value of combat power dies. By initialising them randomly, we don't know who is going to die first.

- Greediness:
  Greediness of the pirate determines whether it will follow the companion's signal for treasure or not. The threshold I have set for greediness is 0.5. The pirate with greediness higher than this, will follow companion and dig the treasure, otherwise it will keep roaming.

This technique is mainly used to keep the decision of the game different every time, and it is making a pirate's behaviour unique.
Using random values of greediness every time, helped me to implement different scenarios where there are 3 cases for a Pirate:

1. The pirate always goes to dig the treasure and collects coins and the other one doesn't. So when they enter attack mode if this pirate has less combat power, all the treasure is transferred to the one who did not even collect anything and vice-versa.
2. The pirate never goes to dig the treasure and collects coins and the other one does. So when they enter attack mode if this pirate has less combat power, no amount of treasure is transferred to the one who killed him and vice-versa.
3. Both of them collect coins and the coins are transferred based on their combat power.

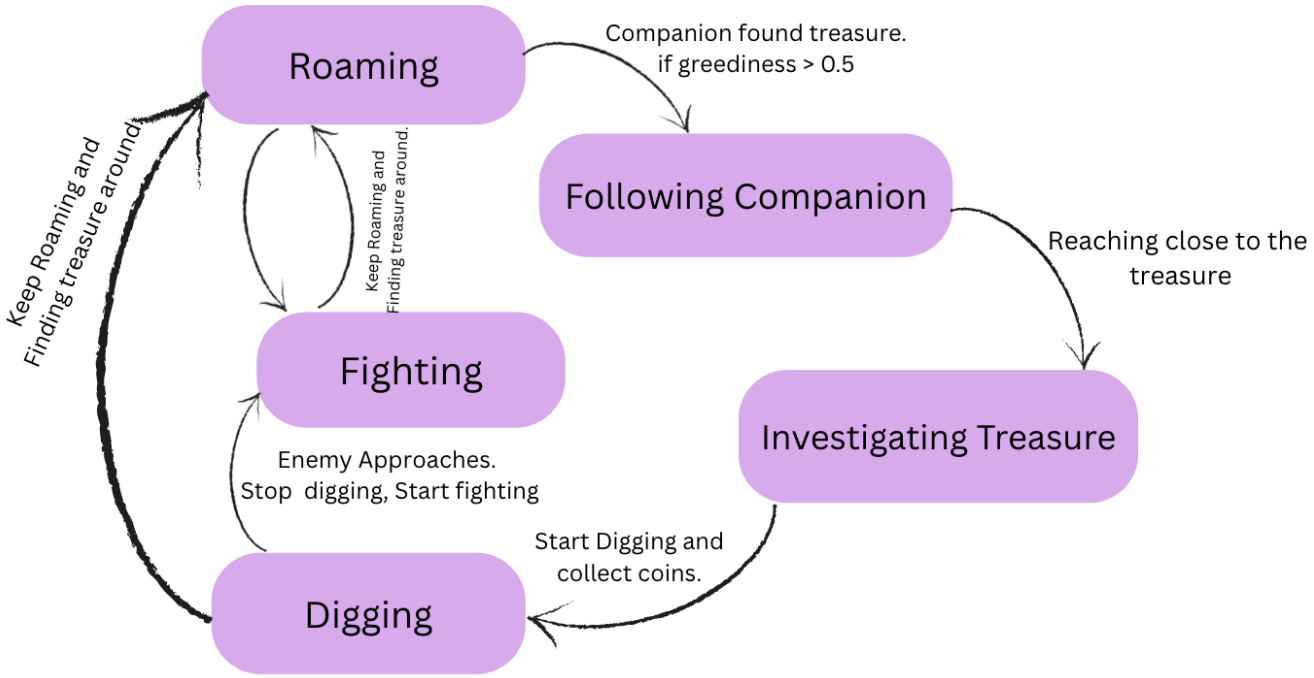
Also, this technique is easy to understand because a pirate's greediness is directly proportional to its will to go after the companion for treasure, otherwise he will keep roaming. This makes the game unpredictable every single time.

This technique is also easy to balance, because I can adjust the range of greediness and threshold to make the scenario more or less challenging.

**Finite State Machines (FSM)** which is used to control pirates' behaviour and actions which influences their decision.

I have used FSM because they are perfect to control the character behavior of the pirate easily. It is feasible to organise different behaviours into states like “Roaming”, “Investigating Treasure”, “Digging” etc. It resulted in a clean and manageable code. Each state has specific rules and transitions which prevents pirates going into any unexpected state and doing impossible or weird things like entering in combat mode while it is digging.
Also, in a game any character has multiple states or modes of behaviour, FSMs match the analogy of how we think of the character’s actions. And it does not require the use of so many if-else and heavy calculations.

Below is the chart to visualise FSM [7].




**NavMesh Integration [8]** is used to solve the pathfinding problem so that pirates and their companions move effortlessly on the terrain path. Each Pirate AI utilises a NavMesh Agent configured to avoid obstacles with inclination more than 20 degrees. It helped them to dynamically avoid static and moving obstacles and handle varying walkable surfaces.

# Reflection

The implementation of above AI techniques helped me demonstrate an unpredictable outcome from the game and handling of those transitions smoothly without any misbehaviour of components.
The randomness in combat power and greediness produced various scenarios because if any non greedy pirate is killed, the other pirate might not have any benefit out of it because they won't be having any coins transferred. This was made more interesting by my teammate, Jai, by adding a logic of random number of coins while initialising treasure chests. All this prevents a predictable winner.

The FSM implementation proved particularly effective in maintaining clean transitions to different states so that no pirate ever engages in incompatible actions like digging during fighting.

**Problems Faced** during implementation:

- Navigation: The most time taking issue was to figure out pathfinding. I have used Unity's NavMesh for this but sometimes characters still get stuck for eg., when a pirate and his companion overlaps at some point.
- Logs and state changing: States were sometimes switching too fast which resulted in improper updation of logs even though the code's working fine with each state executing in a flow.
- Inconsistency with random value: As both the pirates were using the same script, they were having the same random values generated(though it was different most of the time) but that makes the game predictable unnaturally.

**Improvements**
Following improvements could be done to make the game more interesting and better looking:

- Implementation of a health system for both the pirates such that after every attack their health is reduced to some extent rather than them dying.
- Figuring out coordination issues using a better implementation and exploration of Unity's NavMesh for proper navigation.
- Handling text logs properly for a better understanding of what's happening in the game.
- Enhancing UI.

# Conclusion

This report contributes to the details of implementation of my Final Project for this subject: AI for Games. The Pirate AI game successfully demonstrates a varied behaviour game. While working on this, I got to learn how simple AI techniques can create engaging game characters. What I am amazed with the most is how the simple logic and program can make complex-looking behaviour smoothless.

Although there is room for so many improvements, the implementation met core requirements for the project. Looking back, I did a satisfactory job while also solving some technical challenges. However, there were limitations in my approach.

Concluding with what John Carmack said, "*Focus on simplicity and elegance. Fancy algorithms are slow when n is small, and n is usually small"*

# References


[1] https://assetstore.unity.com/packages/2d/textures-materials/floors/outdoor-ground-textures-12555
[2] https://assetstore.unity.com/packages/3d/environments/low-poly-tropical-island-lite-242437
[3] https://assetstore.unity.com/packages/3d/environments/nature-starter-kit-2-52977
[4] https://assetstore.unity.com/packages/3d/characters/humanoids/humans/free-modular-character-fantasy-rpg-human-male-228952
[5] https://assetstore.unity.com/packages/3d/props/weapons/free-pack-of-medieval-weapons-136607
[6] https://www.fab.com/listings/0c8f3917-2461-4775-a853-b995bb93bac5
[7] https://www.canva.com
[8] https://docs.unity3d.com/6000.2/Documentation/ScriptReference/AI.NavMesh.html